

On the Real Part of Butterfly Factorizations, with Application to Fast Transforms

Maël Chaumette, Rémi Gribonval and Elisa Riccietti

Abstract—Fast transforms such as the Discrete Cosine Transform can be expressed as the real part of a product of complex-valued structured sparse matrices known as Kronecker-sparse factors. More generally, products of such structured sparse matrices appear in square dyadic butterfly and monarch matrices, which are known to represent many fast transforms efficiently. Yet, it has remained unclear whether taking the real or the imaginary part of such products preserves this structure. In this letter, we give a condition on the factors to ensure that the structure is preserved. This condition is satisfied in the case of the Discrete Cosine Transform, the Discrete Sine Transform and the Discrete Hartley Transform, allowing us to provide a new representation of these transforms as products of real-valued Kronecker-sparse factors, which opens new avenues for efficient GPU implementations using low-precision numerical format.

Index Terms—Fast transforms, Discrete Cosine Transform, sparse factorization, structured matrices.

I. INTRODUCTION

FAST transforms are a fundamental tool in signal processing and machine learning, enabling efficient computation of linear operations. A given linear operator is usually made fast by expressing it as a product of sparse factors, which can be efficiently stored and implemented. The use of such sparse factorizations dates back to the Fast Fourier Transform (FFT) algorithm [1], which can be interpreted as a factorization into factors with a so-called *butterfly* structure, reducing costs from quadratic to near-linear in the matrix dimension. Butterfly structures also appear in fast transforms such as the Discrete Cosine/Sine/Hartley Transform (DCT/DST/DHT) [2], [3], [4], [5], or the Fast Noiselet transform [6], as well as in hierarchical methods [7], in the fast evaluation of Fourier integral operators [8], in learning or approximating linear operators [9] and in deep learning [10], [11], [12]. Recently, a general class of structured sparse matrices, called Kronecker-sparse factors [13], introduced to generalize the butterfly factorization [14], has been shown to express many families of efficient matrices (see Example 1) while being endowed with provably good factorization algorithms.

The benefits of sparse factorizations in terms of computation time are well-known on CPUs [15], [9]. On GPUs, the situation is more subtle but rapidly evolving: while irregular

sparsity patterns are not well-suited and make the theoretical speed-up difficult to achieve in practice [12], [16], recent works with block structures or dedicated kernels start to close this gap [17], [18]. Furthermore, the availability of low-precision float formats on GPUs offers additional avenues for reducing memory footprint and improving computational efficiency.

Despite becoming standard, low-precision formats on GPUs are however currently not available for complex numbers. As the implementation of many (real-valued) fast transforms such as the DCT, DST or DHT relies on their expression using the real (or the imaginary) part of the FFT, this involves complex-valued Kronecker-sparse factors which cannot benefit from standard low-precision format. This raises the question of whether the real (or imaginary) part of Kronecker-sparse factorizations is still a Kronecker-sparse factorization *with real-valued entries*. To the best of our knowledge, this has not been addressed in the literature. The contribution of this letter is, under a choice of permutation matrices, to give a necessary and sufficient condition on the factors to ensure that the real or the imaginary part of a product of Kronecker-sparse factors is still a product of Kronecker-sparse factors. This allows us to provide a new representation of the DCT, DST and DHT as a factorization with real-valued sparse matrices, unlocking the use of low-precision format.

II. NOTATIONS AND BACKGROUND

We denote by $\text{Re}(\cdot)$ and $\text{Im}(\cdot)$ the real and imaginary parts of a complex number, respectively. Vectors (resp. matrices) are denoted by lowercase (resp. uppercase) bold letters. $\mathbf{1}_{m \times n}$ denotes the matrix of size $m \times n$ with all entries equal to 1. $\mathbf{I}_n$ denotes the identity matrix of size $n \times n$. Finally, for $a \leq b \in \mathbb{N}$, we denote by $\llbracket a, b \rrbracket$ the set of integers $\{a, a+1, \dots, b\}$.

Kronecker-sparse factors are defined by their support, i.e., the set of indices where the matrix has non-zero entries.

Definition 1 (Kronecker-sparse supports and their (sparsity) patterns [13, Definition 4.1]): For $a, b, c, d \in \mathbb{N}$, a Kronecker-sparse factor of (sparsity) pattern $\pi := (a, b, c, d)$ is a matrix in $\mathbb{R}^{abd \times acd}$ or $\mathbb{C}^{abd \times acd}$ such that its support is included in the support of $\mathbf{S}_\pi := \mathbf{I}_a \otimes \mathbf{1}_{b \times c} \otimes \mathbf{I}_d \in \{0, 1\}^{abd \times acd}$. The set of Kronecker-sparse factors with pattern π is denoted by $\mathcal{K}_\pi$.

Fig. 1 shows two examples of Kronecker-sparse factors.

Example 1: Such factors are the basic blocks to build many matrices with efficient implementations:

- *Square dyadic butterflies* [10], [19] are related to many fast transforms, such as the FFT [1] and the Fast Walsh-Hadamard Transform [20], which are $n \times n$ matrices with $n = 2^L$ that can be written in terms of a product $\mathbf{K}_1 \dots \mathbf{K}_L$ where $\mathbf{K}_\ell \in \mathcal{K}_{\pi_\ell}$, $\ell \in \llbracket 1, L \rrbracket$ and $\pi_\ell = (2^{\ell-1}, 2, 2, 2^{L-\ell})$.

Submission date

This project was supported by the SHARP project of the PEPR-IA (ANR-23-PEIA-0008, granted by France 2030), and the project ANR MEPHISTO ANR-24-CE23-7039.

Maël Chaumette and Rémi Gribonval are with Inria, CNRS, ENS de Lyon, Université Claude Bernard Lyon 1, LIP, UMR 5668, 69342, France. (e-mail: mael.chaumette@inria.fr; remi.gribonval@inria.fr)

Elisa Riccietti is with ENS de Lyon, CNRS, Inria, Université Claude Bernard Lyon 1, LIP, UMR 5668, 69342, France. (e-mail: elisa.riccietti@ens-lyon.fr)

- *Monarch factorizations* [17] for weight matrices in transformers (for vision or language tasks) have led to speed-ups of training and inference time. They express an $m \times n$ matrix as a product $\mathbf{K}_1 \mathbf{K}_2$ with $\mathbf{K}_\ell \in \mathcal{K}_{\pi_\ell}$, $\ell = 1, 2$, where $\pi_1 = (1, p, q, m/p)$ and $\pi_2 = (q, m/p, n/q, 1)$ assuming that p, q divide m, n respectively.

General matrix products $\mathbf{K}_1 \cdots \mathbf{K}_L$ with $\mathbf{K}_\ell \in \mathcal{K}_{\pi_\ell}$, $\ell \in \llbracket 1, L \rrbracket$ for some patterns π_ℓ are called *butterfly factorizations* [19] and $(\pi_1, \dots, \pi_L)$ is called the *architecture* of the factorization.

III. MAIN RESULT

In this section, we will prove¹ that, under some condition on the architecture, if a matrix admits a butterfly factorization $\mathbf{A} = \mathbf{K}_1 \cdots \mathbf{K}_L$, then the matrix $\mathbf{A}' := \text{Re}(\mathbf{A})$ also admits a butterfly factorization with a different architecture. We first show that $\text{Re}(\mathbf{A})$ is expressed as a product of L matrices *without* the Kronecker-sparse structure, then we apply well-chosen permutations to recover a Kronecker-sparse factorization.

A. Pre-factorization of the real part

When $L = 1$ (i.e., $\mathbf{A} := \mathbf{K}_1$), as being a Kronecker-sparse factor only depends on the support of $\mathbf{A}$, $\text{Re}(\mathbf{A})$ and $\text{Im}(\mathbf{A})$ are also Kronecker-sparse factors with the same pattern. For $L \geq 2$, simple linear algebra yields $\text{Re}(\mathbf{A}) = \bar{\mathbf{K}}_1 \cdots \bar{\mathbf{K}}_L$ where

$$\begin{aligned} \bar{\mathbf{K}}_1 &:= (\text{Re}(\mathbf{K}_1) \quad \text{Im}(\mathbf{K}_1)) \\ \bar{\mathbf{K}}_\ell &:= \begin{pmatrix} \text{Re}(\mathbf{K}_\ell) & \text{Im}(\mathbf{K}_\ell) \\ -\text{Im}(\mathbf{K}_\ell) & \text{Re}(\mathbf{K}_\ell) \end{pmatrix}, \quad 2 \leq \ell \leq L-1 \\ \bar{\mathbf{K}}_L &:= \begin{pmatrix} \text{Re}(\mathbf{K}_L) \\ -\text{Im}(\mathbf{K}_L) \end{pmatrix}. \end{aligned} \quad (1)$$

However, these factors $\bar{\mathbf{K}}_\ell$, $\ell \in \llbracket 1, L \rrbracket$, are not Kronecker-sparse as illustrated in the middle of Fig. 1 with the support of $\bar{\mathbf{K}}_L$ when $\pi_L = (2, 2, 2, 3)$.

B. Permutations to recover a Kronecker-sparse factorization

Considering any permutation matrices $\mathbf{P}_2, \dots, \mathbf{P}_L$, we have

$$\bar{\mathbf{K}}_1 \cdots \bar{\mathbf{K}}_L = \tilde{\mathbf{K}}_1 \cdots \tilde{\mathbf{K}}_L$$

with the permuted factors

$$\begin{aligned} \tilde{\mathbf{K}}_1 &:= \bar{\mathbf{K}}_1 \mathbf{P}_2^\top \\ \tilde{\mathbf{K}}_\ell &:= \mathbf{P}_\ell \bar{\mathbf{K}}_\ell \mathbf{P}_{\ell+1}^\top, \quad 2 \leq \ell \leq L-1 \\ \tilde{\mathbf{K}}_L &:= \mathbf{P}_L \bar{\mathbf{K}}_L. \end{aligned} \quad (2)$$

Can we find permutations and “maximally sparse” patterns π'_ℓ , $1 \leq \ell \leq L$ such that $\tilde{\mathbf{K}}_\ell \in \mathcal{K}_{\pi'_\ell}$, $1 \leq \ell \leq L$? This is what we investigate next. Quite naturally, based on a count of the number of nonzero entries per row/column of each factor (for example, $\bar{\mathbf{K}}_1$ has twice the number of columns of $\mathbf{K}_1$, and the sparsity of each column is unchanged), we end up targeting the following sparsity patterns:

$$\pi'_\ell := \begin{cases} (a_1, b_1, 2c_1, d_1) & \text{if } \ell = 1 \\ (a_L, 2b_L, c_L, d_L) & \text{if } \ell = L \\ (a_\ell, 2b_\ell, 2c_\ell, d_\ell) & \text{otherwise.} \end{cases} \quad (3)$$

¹ All definitions and proofs of this letter were formalized in the Lean proof assistant. The code is available in the supplementary material.

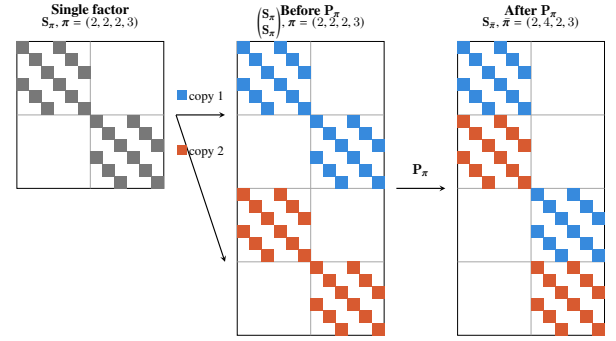


Fig. 1. (Left) Example of the support of a Kronecker-sparse factor with pattern $\pi = (2, 2, 2, 3)$. Here, the non-zero entries are represented by the colored squares. (Middle) The support of the concatenated Kronecker-sparse factors $\begin{pmatrix} \mathbf{S}_\pi \\ \mathbf{S}_\pi \end{pmatrix}$, which is not a Kronecker-sparse factor. (Right) The support of the Kronecker-sparse factor with pattern $\bar{\pi} = (2, 4, 2, 3)$ obtained by applying the permutation matrix $\mathbf{P}_\pi$ on the left to the previous matrix.

C. Finding the right permutation matrices

To satisfy $\tilde{\mathbf{K}}_\ell \in \mathcal{K}_{\pi'_\ell}$ irrespective of the values of the entries of $\mathbf{K}_1, \dots, \mathbf{K}_L$, the permutation matrices $\mathbf{P}_\ell$ have to satisfy:

$$\mathbf{P}_2 \cdot \begin{pmatrix} \mathbf{S}_{\pi_1}^\top \\ \mathbf{S}_{\pi_1} \end{pmatrix} = \mathbf{S}_{(a_1, b_1, 2c_1, d_1)}^\top = \mathbf{S}_{(a_1, 2c_1, b_1, d_1)} \quad (4)$$

$$\mathbf{P}_\ell \cdot \begin{pmatrix} \mathbf{S}_{\pi_\ell} & \mathbf{S}_{\pi_\ell} \\ \mathbf{S}_{\pi_\ell} & \mathbf{S}_{\pi_\ell} \end{pmatrix} \cdot \mathbf{P}_{\ell+1}^\top = \mathbf{S}_{(a_\ell, 2b_\ell, 2c_\ell, d_\ell)}, \quad 2 \leq \ell \leq L-1 \quad (5)$$

$$\mathbf{P}_L \cdot \begin{pmatrix} \mathbf{S}_{\pi_L} \\ \mathbf{S}_{\pi_L} \end{pmatrix} = \mathbf{S}_{(a_L, 2b_L, c_L, d_L)}. \quad (6)$$

We will use a particular choice of permutation matrices to satisfy (4), (5) and (6). Potentially, other choices of permutation matrices could also satisfy these conditions, but we do not explore this question in this letter. We propose to use $\mathbf{P}_\ell := \mathbf{P}_{\pi_\ell}$ where, given any pattern $\pi = (a, b, c, d)$, we define

$$\mathbf{P}_\pi := \mathbf{C}_{a,2} \otimes \mathbf{I}_{bd} \quad (7)$$

with $\mathbf{C}_{m,n} \in \mathbb{R}^{mn \times mn}$ the so-called *commutation matrix* [21], also known as (m, n) -*perfect shuffle* [22].

Definition 2 (Commutation matrix [21]): The commutation matrix $\mathbf{C}_{m,n}$ is the permutation matrix such that, for any matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$, $\mathbf{C}_{m,n} \text{vec}(\mathbf{M}) = \text{vec}(\mathbf{M}^\top)$, where $\text{vec}(\cdot)$ denotes the vectorization operator stacking the columns of $\mathbf{M}$. It corresponds to the following permutation σ of the set $\llbracket 1, mn \rrbracket$

$$\sigma(i + m(j-1)) = j + n(i-1), \quad i \in \llbracket 1, m \rrbracket, j \in \llbracket 1, n \rrbracket.$$

In particular $\mathbf{C}_{a,2}$ applied to a matrix permutes its rows by grouping on top the rows of indices $i = 0$ and $i = a$, then the rows of indices $i = 1$ and $i = a+1$, and so on. By construction, the matrix $\mathbf{P}_\pi$ acts similarly but on groups of bd consecutive rows instead of single rows. Hence, applying $\mathbf{P}_\pi$ on the left to the concatenated Kronecker-sparse factors appearing in (4), (5) and (6) with pattern π will group the rows to obtain a Kronecker-sparse factor with pattern $\bar{\pi} := (a, 2b, c, d)$. This is illustrated in Fig. 1 for $\pi = (2, 2, 2, 3)$ and formalized in the following lemma.

Lemma 1: Given $\pi = (a, b, c, d)$, the matrix $\mathbf{P}_\pi$ satisfies

$$\mathbf{P}_\pi \cdot \begin{pmatrix} \mathbf{S}_\pi \\ \mathbf{S}_\pi \end{pmatrix} = \mathbf{S}_{\bar{\pi}} \in \mathbb{R}^{2abd \times acd},$$

where $\bar{\pi} := (a, 2b, c, d)$.

Proof: A key property [21, Theorem 3.1] of $\mathbf{C}_{m,n}$ is the identity $\mathbf{A} \otimes \mathbf{B} = \mathbf{C}_{m,n}(\mathbf{B} \otimes \mathbf{A})\mathbf{C}_{p,q}^\top$ for every $\mathbf{A} \in \mathbb{C}^{m \times p}$ and $\mathbf{B} \in \mathbb{C}^{n \times q}$, with $\otimes$ the Kronecker product. Since $\mathbf{C}_{p,1} = \mathbf{I}_p$ for every p we deduce when specializing to $m = a$, $n = 2$ and $q = 1$ that

$$\mathbf{A} \otimes \mathbf{1}_{2 \times 1} = \mathbf{C}_{a,2}(\mathbf{1}_{2 \times 1} \otimes \mathbf{A}), \quad \forall \mathbf{A} \in \mathbb{C}^{a \times p}. \quad (8)$$

Combined with Definition 1, (7) and the mixed-product property $((\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD}))$ for appropriate dimensions [22]) of the Kronecker product, we obtain

$$\begin{aligned} \mathbf{P}_\pi \cdot \begin{pmatrix} \mathbf{S}_\pi \\ \mathbf{S}_\pi \end{pmatrix} &= (\mathbf{C}_{a,2} \otimes \mathbf{I}_{bd}) \cdot (\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_a \otimes \mathbf{1}_{b \times c} \otimes \mathbf{I}_d) \\ &= (\mathbf{C}_{a,2} \cdot (\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_a)) \otimes (\mathbf{I}_{bd}(\mathbf{1}_{b \times c} \otimes \mathbf{I}_d)) \\ &= (\mathbf{I}_a \otimes \mathbf{1}_{2 \times 1}) \otimes (\mathbf{1}_{b \times c} \otimes \mathbf{I}_d) \\ &= \mathbf{I}_a \otimes \underbrace{(\mathbf{1}_{2 \times 1} \otimes \mathbf{1}_{b \times c})}_{=\mathbf{1}_{2b \times c}} \otimes \mathbf{I}_d = \mathbf{S}_{\bar{\pi}}. \quad \blacksquare \end{aligned}$$

We now characterize when $\mathbf{P}_\ell := \mathbf{P}_{\pi_\ell}$ satisfy (4)-(5)-(6).

Lemma 2: For any architecture $(\pi_1, \dots, \pi_L)$ with $\pi_\ell = (a_\ell, b_\ell, c_\ell, d_\ell)$, $\ell \in \llbracket 1, L \rrbracket$, (6) is satisfied with $\mathbf{P}_L := \mathbf{P}_{\pi_L}$. Besides, (4) and (5) hold with $\mathbf{P}_\ell := \mathbf{P}_{\pi_\ell}$ if, and only if, for every $\ell \in \llbracket 1, L-1 \rrbracket$, we have

$$\begin{aligned} &(\mathbf{C}_{a_{\ell+1},2} \otimes \mathbf{I}_{b_{\ell+1}d_{\ell+1}}) \cdot \\ &\left(\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell} \right) = \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{2c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}, \end{aligned} \quad (9)$$

where $b'_\ell = b_\ell$ if $\ell = 1$ and $b'_\ell = 2b_\ell$ otherwise.

Proof: First, as a consequence of Lemma 1, we have

$$\mathbf{P}_{\pi_\ell} \cdot \begin{pmatrix} \mathbf{S}_{\pi_\ell} \\ \mathbf{S}_{\pi_\ell} \end{pmatrix} = \mathbf{S}_{\bar{\pi}_\ell}, \quad 2 \leq \ell \leq L,$$

so (6) holds with $\mathbf{P}_L := \mathbf{P}_{\pi_L}$. Next, (4) rewrites (for $\ell = 1$) as

$$\mathbf{P}_{\pi_{\ell+1}} \cdot \begin{pmatrix} \mathbf{S}_{(a_\ell, c_\ell, b_\ell, d_\ell)} \\ \mathbf{S}_{(a_\ell, c_\ell, b_\ell, d_\ell)} \end{pmatrix} = \mathbf{S}_{(a_\ell, 2c_\ell, b_\ell, d_\ell)},$$

which is equivalent to (9) for $\ell = 1$ by Definition 1 and (7). Now, denoting $\bar{\pi} := (a, 2b, 2c, d)$ for $\pi := (a, b, c, d)$, since

$$\mathbf{P}_{\pi_\ell} \cdot \begin{pmatrix} \mathbf{S}_{\pi_\ell} & \mathbf{S}_{\pi_\ell} \\ \mathbf{S}_{\pi_\ell} & \mathbf{S}_{\pi_\ell} \end{pmatrix} = (\mathbf{S}_{\bar{\pi}_\ell} \quad \mathbf{S}_{\bar{\pi}_\ell}), \quad 2 \leq \ell \leq L,$$

we get that (5) is equivalent to $(\mathbf{S}_{\bar{\pi}_\ell} \quad \mathbf{S}_{\bar{\pi}_\ell}) \cdot \mathbf{P}_{\pi_{\ell+1}}^\top = \mathbf{S}_{\bar{\pi}_\ell}^\top$, $2 \leq \ell \leq L-1$, which can be rewritten as

$$\mathbf{P}_{\pi_{\ell+1}} \cdot \begin{pmatrix} \mathbf{S}_{(a_\ell, c_\ell, 2b_\ell, d_\ell)} \\ \mathbf{S}_{(a_\ell, c_\ell, 2b_\ell, d_\ell)} \end{pmatrix} = \mathbf{S}_{(a_\ell, 2c_\ell, 2b_\ell, d_\ell)}, \quad 2 \leq \ell \leq L-1,$$

or equivalently as (9) for $2 \leq \ell \leq L-1$. $\blacksquare$

This lemma turns conditions (4), (5), and (6) on the permutation matrices into (9), a condition on the architecture of the Kronecker-sparse factors. To conclude, we turn this condition into three explicit conditions on the Kronecker-sparse patterns.

Proposition 1: Consider an architecture $(\pi_1, \dots, \pi_L)$. The following are equivalent:

- for every $\ell \in \llbracket 1, L-1 \rrbracket$ we have

- C1. $a_\ell c_\ell d_\ell = a_{\ell+1} b_{\ell+1} d_{\ell+1}$;
- C2. $d_\ell | b_{\ell+1} d_{\ell+1}$;
- C3. $a_\ell | a_{\ell+1}$.

- defining $\bar{\mathbf{K}}_\ell$ as in (2) with $\mathbf{P}_\ell := \mathbf{P}_{\pi_\ell}$ as in (7) and $\mathbf{K}_\ell \in \mathcal{K}_{\pi_\ell}$, we have $\bar{\mathbf{K}}_\ell \in \mathcal{K}_{\pi'_\ell}$, $1 \leq \ell \leq L$ with π'_ℓ as in (3).

Proof: C1 is necessary and sufficient to ensure compatible matrix dimensions for matrix multiplication.

Sufficiency of C1-C2-C3. We denote $r := \frac{b_{\ell+1}d_{\ell+1}}{d_\ell}$, which by C2 is an integer and by C1 is equal to $\frac{a_\ell c_\ell}{a_{\ell+1}}$. Moreover, as $a_\ell | a_{\ell+1}$ by C3, there exists an integer p such that $p := \frac{a_{\ell+1}}{a_\ell}$, so $c_\ell = pr$. Hence, we can decompose the matrix $\mathbf{1}_{c_\ell \times b'_\ell} = \mathbf{1}_{p \times b'_\ell} \otimes \mathbf{1}_{r \times 1}$ then use the mixed-product property of the Kronecker product, therefore the left-hand side of (9) can be rewritten as

$$\begin{aligned} &(\mathbf{C}_{a_{\ell+1},2} \otimes \mathbf{I}_{d_{\ell+1}b_{\ell+1}}) \cdot \left(\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell} \right) \\ &= (\mathbf{C}_{a_{\ell+1},2} \otimes \mathbf{I}_{d_{\ell+1}b_{\ell+1}}) \cdot \left(\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{p \times b'_\ell} \otimes \mathbf{1}_{r \times 1} \otimes \mathbf{I}_{d_\ell} \right) \\ &= \mathbf{C}_{a_{\ell+1},2} \cdot \left(\mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{p \times b'_\ell} \right) \otimes (\mathbf{I}_{d_{\ell+1}b_{\ell+1}} \cdot (\mathbf{1}_{r \times 1} \otimes \mathbf{I}_{d_\ell})) \\ &= \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{p \times b'_\ell} \otimes \mathbf{1}_{2 \times 1} \otimes \mathbf{1}_{r \times 1} \otimes \mathbf{I}_{d_\ell} \\ &= \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{2p \times b'_\ell} \otimes \mathbf{1}_{r \times 1} \otimes \mathbf{I}_{d_\ell} = \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{2c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}. \end{aligned}$$

Necessity of C2&C3. With $\mathbf{X}_\ell := \mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}$, $\mathbf{Y}_\ell := \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{2c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}$, $\ell \in \llbracket 1, L-1 \rrbracket$, (9) reads $\mathbf{P}_{\pi_{\ell+1}} \cdot \mathbf{X}_\ell = \mathbf{Y}_\ell$. We continue by indexing the rows of $\mathbf{X}_\ell$ by $(\varepsilon, i, k_1, k_2) \in \{0, 1\} \times \llbracket 0, a_\ell - 1 \rrbracket \times \llbracket 0, c_\ell - 1 \rrbracket \times \llbracket 0, d_\ell - 1 \rrbracket$ via $\text{id}_x(\varepsilon, i, k_1, k_2) := \varepsilon(a_\ell c_\ell d_\ell) + i(c_\ell d_\ell) + k_1 d_\ell + k_2$, and the columns of $\mathbf{X}_\ell$ (equivalently of $\mathbf{Y}_\ell$ because they live in the same column space of size $a_\ell b'_\ell d_\ell$) by an index defined using $(i', j, k'_2) \in \llbracket 0, a_\ell - 1 \rrbracket \times \llbracket 0, b'_\ell - 1 \rrbracket \times \llbracket 0, d_\ell - 1 \rrbracket$ similarly. Since $\mathbf{X}_\ell = \mathbf{1}_{2 \times 1} \otimes \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}$

$$\mathbf{X}_\ell[(\varepsilon, i, k_1, k_2), (i', j, k'_2)] = \delta_{i,i'} \delta_{k_2, k'_2}.$$

We now index the rows of $\mathbf{Y}_\ell$ by $(i, k'_1, k_2) \in \llbracket 0, a_\ell - 1 \rrbracket \times \llbracket 0, 2c_\ell - 1 \rrbracket \times \llbracket 0, d_\ell - 1 \rrbracket$ via $\text{id}_x'(i, k'_1, k_2) := i(2c_\ell d_\ell) + k'_1 d_\ell + k_2$. Since $\mathbf{Y}_\ell = \mathbf{I}_{a_\ell} \otimes \mathbf{1}_{2c_\ell \times b'_\ell} \otimes \mathbf{I}_{d_\ell}$, we have

$$\mathbf{Y}_\ell[(i, k'_1, k_2), (i', j, k'_2)] = \delta_{i,i'} \delta_{k_2, k'_2}.$$

In particular, to each fixed (i, k_2) we can associate a *distinct* row vector $\mathbf{v}_{i, k_2}$ such that all rows of $\mathbf{X}_\ell$ with second/last coordinates (i, k_2) (there are $2c_\ell$ of them indexed by (ε, k_1)) and all rows of $\mathbf{Y}_\ell$ with first/last coordinates (i, k_2) (there are $2c_\ell$ of them indexed by k'_1) are equal to $\mathbf{v}_{i, k_2}$. Hence, denoting by ρ_π the row-permutation associated to $\mathbf{P}_\pi$, the equality $\mathbf{P}_{\pi_{\ell+1}} \mathbf{X}_\ell = \mathbf{Y}_\ell$ implies (and is in fact equivalent to)

$$\begin{aligned} &\rho_{\pi_{\ell+1}} \left(\{ \text{id}_x(\varepsilon, i, k_1, k_2) \}_{\varepsilon \in \{0,1\}, k_1 \in \llbracket 0, c_\ell - 1 \rrbracket} \right) \\ &= \{ \text{id}_x'(i, k'_1, k_2) \}_{k'_1 \in \llbracket 0, 2c_\ell - 1 \rrbracket}, \\ &\forall (i, k_2) \in \llbracket 0, a_\ell - 1 \rrbracket \times \llbracket 0, d_\ell - 1 \rrbracket. \end{aligned} \quad (10)$$

To prove C2, we use (10) with $i = k_2 = 0$. As the integers in $\{ \text{id}_x'(0, k'_1, 0) \}_{k'_1 \in \llbracket 0, 2c_\ell - 1 \rrbracket}$ are multiples of d_ℓ , d_ℓ divides $\rho_{\pi_{\ell+1}}(\text{id}_x(\varepsilon, 0, k_1, 0))$. For $\varepsilon = 1, k_1 = 0$, since $\text{id}_x(1, 0, 0, 0) = a_\ell c_\ell d_\ell = a_{\ell+1} b_{\ell+1} d_{\ell+1}$ (by C1), this means that d_ℓ divides $\rho_{\pi_{\ell+1}}(a_{\ell+1} b_{\ell+1} d_{\ell+1})$. By definition of $\mathbf{P}_{\pi_{\ell+1}}$ we have $\rho_{\pi_{\ell+1}}(a_{\ell+1} b_{\ell+1} d_{\ell+1}) = b_{\ell+1} d_{\ell+1}$, this establishes C2.

C3 trivially holds when $a_\ell = 1$. When $a_\ell \geq 2$, we use (10) with $i = 1, \varepsilon = k_1 = k_2 = 0$. As $\text{id}\mathbf{x}(0, 1, 0, 0) = c_\ell d_\ell$, we obtain as above that $\rho_{\pi_{\ell+1}}(c_\ell d_\ell) = (2c_\ell + k'_1)d_\ell$ for some $k'_1 \in \llbracket 0, 2c_\ell - 1 \rrbracket$. By definition of $\mathbf{P}_{\pi_{\ell+1}}$, $\rho_{\pi_{\ell+1}}(c_\ell d_\ell)$ depends on the Euclidean division of $c_\ell d_\ell$ by $b_{\ell+1}d_{\ell+1}$. By C2, $r := b_{\ell+1}d_{\ell+1}/d_\ell$ is an integer, and using the Euclidean division $c_\ell = pr + q$, $p, q \in \mathbb{N}$, $0 \leq q < r$, with C1, we get $c_\ell d_\ell = pb_{\ell+1}d_{\ell+1} + qd_\ell$ and $qd_\ell < rd_\ell = b_{\ell+1}d_{\ell+1}$. It follows that $\rho_{\pi_{\ell+1}}(c_\ell d_\ell) = (2pr + q)d_\ell$, hence $(2c_\ell + k'_1)d_\ell = (2pr + q)d_\ell$, i.e., $2c_\ell + k'_1 = 2pr + q = 2(c_\ell - q) + q = 2c_\ell - q$. Thus, $k'_1 = q = 0$, hence $r|c_\ell$, and since by C1 $r = a_\ell c_\ell / a_{\ell+1}$, we conclude that $a_\ell | a_{\ell+1}$. This establishes C3. ■

D. Consequences

A first direct consequence of this proposition is the following corollary showing that every chainable architecture² satisfies the three conditions of the proposition.

Corollary 1: If a matrix $\mathbf{A}$ admits a Kronecker-sparse factorization with a chainable architecture [13, Definition 4.12], then $\text{Re}(\mathbf{A})$ also admits a Kronecker-sparse factorization with the new architecture $(\pi'_1, \dots, \pi'_L)$ where π'_ℓ is as in (3).

The chainability condition allows to obtain error guarantees on the Kronecker-sparse factorizations performed in [13], which is a significant advantage in practical applications.

A similar result holds for the imaginary part since $\text{Im}(\mathbf{K}_1 \dots \mathbf{K}_L) = \text{Re}((-j\mathbf{K}_1) \dots \mathbf{K}_L)$ and $j\mathbf{K}_1$ is Kronecker-sparse with the same pattern as $\mathbf{K}_1$. In fact, the same holds for $\text{Re}(z\mathbf{K}_1 \dots \mathbf{K}_L)$ for any $z \in \mathbb{C}$.

A more concrete consequence is the existence of a fully real-valued Kronecker-sparse implementation of the DCT, DST, and DHT when the dimension is a power of two. It is indeed known [13] (and easy to check) that the square dyadic butterfly factors appearing in the FFT (and the monarch factors) are a chainable architecture, and one can express [3] the DCT/DST matrices as $\mathbf{L} \text{Re}(\mathbf{D}\mathbf{F})\mathbf{R}$, where $\mathbf{L}, \mathbf{R}$ are real-valued, $\mathbf{D} \in \{\mathbf{D}_C, \mathbf{D}_S\}$ is diagonal, and $\mathbf{F}$ is a DFT matrix, which can be written as $\mathbf{F} = \mathbf{B}\mathbf{P}$ with $\mathbf{B}$ a complex matrix with square dyadic butterfly factorization and $\mathbf{P}$ the so-called bit-reversal permutation. As a result the DCT and the DST write as $\mathbf{L} \text{Re}(\mathbf{D}\mathbf{B})\mathbf{P}\mathbf{R}$ and we conclude since $\mathbf{D}\mathbf{B}$ still has a square dyadic butterfly factorization. Since the DHT is the a sum of the DCT and the DST, it similarly writes $\mathbf{L} \text{Re}((\mathbf{D}_C + \mathbf{D}_S)\mathbf{B})\mathbf{P}\mathbf{R}$ which admits a Kronecker-sparse factorization for the same reasons.

IV. EXPERIMENTS

Finally, we perform some experiments to evaluate the inference time of the DCT applied to a vector $\mathbf{x} \in \mathbb{R}^{n \times b}$, where b is the batch size, with the real-valued Kronecker-sparse factorization. Considering a batch of vectors is relevant in practice, since the DCT is rarely applied to a single vector but rather to large batches of data: for instance, image compression standards such as JPEG apply the DCT to every 8×8 block of an image [23], and audio coders apply the DCT to every frame

²This condition is sufficient to ensure that the product of two Kronecker-sparse factors is also a Kronecker-sparse factor. This condition corresponds to $\text{C1} \wedge \text{C2} \wedge \text{C3} \wedge (d_{\ell+1} | d_\ell, \forall \ell)$.

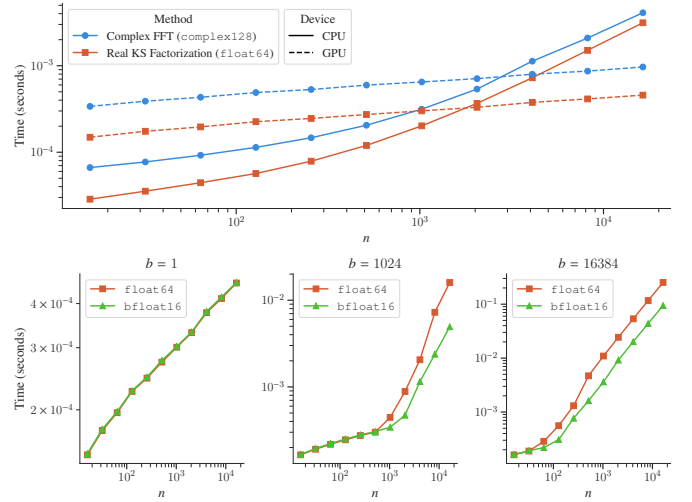


Fig. 2. Inference time of the DCT applied to a vector $\mathbf{x} \in \mathbb{R}^{n \times b}$ in terms of n . (Up) Comparison, on CPU and GPU, with $b = 1$ of two implementations: the one provided by [3] based on the complex-valued FFT and the real-valued factorization proposed in this letter. (Down) Real-valued factorization on GPU for different batch sizes b and different floating-point formats.

of a signal [24]. The code is available in the supplementary material.

As illustrated in Fig. 2 (Up), on CPU and GPU, we observe that the real-valued factorization is faster for every size of the input (e.g., 2 times faster on GPU), in terms of computation time, than the one provided by [3], which performs a complex-valued discrete Fourier transform.

Moreover our factorization is based on real-valued matrices which, on GPUs, can be computed faster with standard low-precision formats such as `float16` or `bfloat16`. This difference is more pronounced when the batch size is increased. Indeed, as illustrated in Fig. 2 (Down), while the inference time in `bfloat16` is equivalent to the one in `float64` when $b = 1$, for larger b there is a dimension threshold beyond which the inference time in `bfloat16` is significantly reduced compared to `float64`. Since naive rounding to such lower-precision formats (as done in these experiments) comes with some loss of accuracy, recent works have developed methods to mitigate this impact and improve over naive rounding [25].

V. CONCLUSION

In this letter, we addressed the question of whether the real or the imaginary part of a product of structured-sparse matrices, called Kronecker-sparse factors, retains the same structure. We provided, under a choice of permutation matrices, a necessary and sufficient condition on the factors of the product to ensure this property. As a direct consequence, we proved that the Discrete Cosine/Sine/Hartley Transforms all admit a real-valued Kronecker-sparse factorization, which, to the best of our knowledge, is a new result in the literature. This result is of practical interest and has been implemented in the Python library `lazylinop`³ for the design of efficient algorithms for these transforms.

³<https://lazylinop.inria.fr>

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] N. Ahmed, T. Natarajan, and K. Rao, "Discrete cosine transform," *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, 1974.
- [3] J. Makhoul, "A fast cosine transform in one and two dimensions," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, pp. 27–34, 1980.
- [4] R. Bracewell, "The fast hartley transform," *Proceedings of the IEEE*, vol. 72, no. 8, pp. 1010–1018, 1984.
- [5] H. Sorensen, D. Jones, C. Burrus, and M. Heideman, "On computing the discrete hartley transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 5, pp. 1231–1238, 1985.
- [6] R. Coifman, F. Geshwind, and Y. Meyer, "Noiselets," *Applied and Computational Harmonic Analysis*, vol. 10, no. 1, pp. 27–44, 2001.
- [7] E. Michielssen and A. Boag, "A multilevel matrix decomposition algorithm for analyzing scattering from large structures," *IEEE Transactions on Antennas and Propagation*, vol. 44, no. 8, pp. 1086–1093, 1996.
- [8] E. Candès, L. Demanet, and L. Ying, "A fast butterfly algorithm for the computation of fourier integral operators," *Multiscale Modeling & Simulation*, vol. 7, no. 4, pp. 1727–1750, 2009.
- [9] L. Le Magoarou and R. Gribonval, "Flexible multilayer sparse approximations of matrices and applications," *IEEE Journal of Selected Topics in Signal Processing*, vol. 10, no. 4, pp. 688–700, 2016.
- [10] T. Dao, A. Gu, M. Eichhorn, A. Rudra, and C. Ré, "Learning fast algorithms for linear transforms using butterfly factorizations," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97. PMLR, 2019, pp. 1517–1527.
- [11] T. Dao, N. S. Sohoni, A. Gu, M. Eichhorn, A. Blonder, M. Leszczynski, A. Rudra, and C. Ré, "Kaleidoscope: An efficient, learnable representation for all structured linear maps," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.
- [12] B. Chen, T. Dao, K. Liang, J. Yang, Z. Song, A. Rudra, and C. Ré, "Pixelated butterfly: Simple and efficient sparse training for neural network models," in *International Conference on Learning Representations*, 2021.
- [13] Q.-T. Le, L. Zheng, E. Riccietti, and R. Gribonval, "Butterfly factorization with error guarantees," *SIAM Journal on Matrix Analysis and Applications*, vol. 46, no. 4, pp. 2253–2309, 2025.
- [14] Y. Li, H. Yang, E. R. Martin, K. L. Ho, and L. Ying, "Butterfly factorization," *Multiscale Modeling & Simulation*, vol. 13, no. 2, pp. 714–732, 2015.
- [15] M. Puschel, J. Moura, J. Johnson, D. Padua, M. Veloso, B. Singer, J. Xiong, F. Franchetti, A. Gacic, Y. Voronenko, K. Chen, R. Johnson, and N. Rizzolo, "Spiral: Code generation for dsp transforms," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 232–275, 2005.
- [16] T. Hoefer, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks," *J. Mach. Learn. Res.*, vol. 22, no. 1, Jan. 2021.
- [17] T. Dao, B. Chen, N. S. Sohoni, A. Desai, M. Poli, J. Grogan, A. Liu, A. Rao, A. Rudra, and C. Re, "Monarch: Expressive structured matrices for efficient and accurate training," in *Proceedings of the 39th International Conference on Machine Learning*, vol. 162. PMLR, 2022, pp. 4690–4721.
- [18] A. Gonon, L. Zheng, P. Carrivain, and T.-Q. Le, "Fast inference with Kronecker-sparse matrices," in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267. PMLR, 2025, pp. 20075–20102.
- [19] L. Zheng, E. Riccietti, and R. Gribonval, "Efficient identification of butterfly sparse matrix factorizations," *SIAM Journal on Mathematics of Data Science*, vol. 5, no. 1, pp. 22–49, 2023.
- [20] A. Fino and V. R. Algazi, "Unified matrix treatment of the fast walsh-hadamard transform," *IEEE Transactions on Computers*, vol. 100, no. 11, pp. 1142–1146, 1976.
- [21] J. R. Magnus and H. Neudecker, "The commutation matrix: Some properties and applications," *The Annals of Statistics*, vol. 7, no. 2, pp. 381–394, 1979.
- [22] C. F. Van Loan, "The ubiquitous kronecker product," *Journal of computational and applied mathematics*, vol. 123, no. 1-2, pp. 85–100, 2000.
- [23] G. Wallace, "The jpeg still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.
- [24] J. Princen and A. Bradley, "Analysis/synthesis filter bank design based on time domain aliasing cancellation," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 34, no. 5, pp. 1153–1161, 1986.
- [25] R. Gribonval, T. Mary, and E. Riccietti, "Optimal quantization of rank-one matrices in floating-point arithmetic—with applications to butterfly factorizations," 2023, working paper or preprint.